

Schema Evolution Compatibility Matrix

A Reference Guide for Event Streaming and Schema Registries (Apicurio / Confluent)

When evolving data structures in event-driven microservices architectures, compatibility rules dictate how schema registries enforce structural safety across distributed boundaries. This matrix acts as a guide to prevent producers or consumers from breaking during version changes.

Compatibility Mode	Who Can Read What?	What changes are usually allowed?	Who should you upgrade first?
Backward	New schema versions can read data written by Old schemas.	• Add fields with default values. • Delete optional fields.	Consumers first
Forward	Old schema versions can read data written by New schemas.	• Add new fields (ignored by old code). • Delete fields that had default values.	Producers first
Full	Both : Old reads New, and New reads Old data seamlessly.	• Add or delete fields that have predefined default values.	Either order (No deployment constraint)

KEY ARCHITECTURAL INVARIANTS

The Golden Rule of Defaults: In a schema-centric architecture (such as one using Avro or Protobuf), almost all compatibility transitions require strict usage of default values. Without a defined default value, a new reader cannot fill the structural gap when missing fields are encountered in older data payloads, and older readers cannot reconcile structural deletions.

Deployment Strategy Tip: If central engineering maintains a core data topic but downstream teams consume asynchronously at their own pace, enforcing **Forward Compatibility** avoids blocking producer updates while waiting for cross-functional downstream coordination.